C++ COMPLETE REFERENCE LIBRARY • MODULE 17

# 17 FILE HANDLING — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

## Module Table of Contents


1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 fstream classes and modes
3. **Theory Chapter:** 02 file pointers seekg seekp tellg tellp
4. **Theory Chapter:** 03 text vs binary files and eof
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions


# 17 — File Handling

This module covers `<fstream>` file streams ( `ofstream` , `ifstream` , `fstream` ), file opening modes ( `ios::in` , `ios::out` , `ios::app` , `ios::binary` ), file position pointers ( `seekg` , `seekp` , `tellg` , `tellp` ), and reading/writing classroom objects.

## Directory Contents

- **Theory ( `01_THEORY/` )**:
  - `01_fstream_classes_and_modes.md` (./01_THEORY/01_fstream_classes_and_modes.md)
  - `02_file_pointers_seekg_seekp_tellg_tellp.md` (./01_THEORY/02_file_pointers_seekg_seekp_tellg_tellp.md)
  - `03_text_vs_binary_files_and_eof.md` (./01_THEORY/03_text_vs_binary_files_and_eof.md)
- **Programs ( `02_PROGRAMS/` )**:
  - `01_write_and_read_text_file.cpp` (./02_PROGRAMS/01_write_and_read_text_file.cpp) (Category A)
  - `02_file_pointers_seekg_tellg.cpp` (./02_PROGRAMS/02_file_pointers_seekg_tellg.cpp) (Category A)

  - `03_binary_file_student_record.cpp` (./02_PROGRAMS/03_binary_file_student_record.cpp) (Category A)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_fstream_hierarchy.svg` (./05_DIAGRAMS/diagram_01_fstream_hierarchy.svg)

# 01 — File Stream Classes (`ofstream`, `ifstream`, `fstream`) and Opening Modes




fstream Hierarchy

1. **`std::ofstream`**: Output file stream (for writing to files; default mode `ios::out | ios::trunc`).
2. **`std::ifstream`**: Input file stream (for reading from files; default mode `ios::in`).
3. **`std::fstream`**: Bidirectional file stream (default mode `ios::in | ios::out`).

| Mode Flag | Meaning |
|---|---|
| `ios::in` | Open for reading |
| `ios::out` | Open for writing |
| `ios::app` | Append all writes to the end of the file |
| `ios::ate` | Move initial file position to the end upon opening |
| `ios::trunc` | Discard existing file contents if the file already exists |
| `ios::binary` | Open in binary mode (no newline character translation) |

## 02 — File Position Pointers: `seekg`, `seekp`, `tellg`, and `tellp`

- `g` **stands for "get" (input /** `ifstream` **)**, and `p` **stands for "put" (output /** `ofstream` **)**:
  - `tellg()` / `tellp()` : Returns the current byte offset ( `streampos` ) of the get/put pointer.
  - `seekg(offset, direction)` / `seekp(offset, direction)` : Moves the get/put pointer relative to `ios::beg` (beginning), `ios::cur` (current position), or `ios::end` (end of file).

# 03 — Text vs. Binary Files and Why `while (!fin.eof())` is an Anti-Pattern

## Why `while (!fin.eof())` Reads the Last Line Twice

`eofbit` is set **only after** a read operation tries to read past the end of the file and fails! If you loop on `while (!fin.eof()) { fin >> x; cout << x; }`, after reading the last item, `eof()` is still `false`; the loop runs one extra time, `fin >> x` fails, and `cout << x` prints the previous value of `x` a second time!

- **Correct Idiom**: Loop directly on the read expression: `while (fin >> x)` or `while (getline(fin, line))`.

# Visual Concept & Memory Diagrams (17 FILE HANDLING)




Diagram: diagram 01 fstream hierarchy

# Annotated C++ Programs (17 FILE HANDLING)

## Program: `01_write_and_read_text_file.cpp`

**C++ SOURCE FILE • 01_write_and_read_text_file.cpp**

```cpp
// Program Title: Writing to and Reading from a Text File Using ofstream and ifstream
// Topic: File Handling — Text Files
// Purpose: Demonstrate writing student data to a file and reading it back safely using
while(getline(...)).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_fstream_classes_and_modes.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    {
        ofstream fout("notes_sample.txt");
        fout << "Roll 101: 95" << endl;
        fout << "Roll 102: 88" << endl;
    } // fout automatically closes via RAII destructor here

    ifstream fin("notes_sample.txt");
    string line;
    while (getline(fin, line)) {
        cout << line << endl;
    }

    return 0;
}

/*
Expected Deterministic Output:
Roll 101: 95
Roll 102: 88
*/
```

## Program: `02_file_pointers_seekg_tellg.cpp`

**C++ SOURCE FILE • 02_file_pointers_seekg_tellg.cpp**

```cpp
// Program Title: Random File Access Using seekg() and tellg()
// Topic: File Handling — File Position Pointers
// Purpose: Demonstrate measuring file size with seekg(0, ios::end) and tellg().
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_file_pointers_seekg_seekp_tellg_tellp.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    {
        ofstream fout("data_bytes.txt", ios::out | ios::trunc);
        fout << "ABCDEFGH";
    }

    ifstream fin("data_bytes.txt");
    fin.seekg(0, ios::end);
    cout << "Total bytes in file: " << fin.tellg() << endl;

    fin.seekg(3, ios::beg);
    char ch;
    fin.get(ch);
    cout << "Character at offset 3: " << ch << endl;

    return 0;
}

/*
Expected Deterministic Output:
Total bytes in file: 8
Character at offset 3: D
*/
```

## Program: `03_binary_file_student_record.cpp`

**C++ SOURCE FILE • 03_binary_file_student_record.cpp**

```cpp
// Program Title: Binary File Read and Write of a Classroom Pod Struct/Class
// Topic: File Handling — Binary Files (write and read)
// Purpose: Demonstrate binary serialization of a trivially-copyable classroom student
record.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_text_vs_binary_files_and_eof.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <fstream>
using namespace std;

class student_record {
public:
    int roll;
    int marks;
};

int main() {
    student_record s_out{105, 97};
    {
        ofstream fout("student.bin", ios::binary);
        fout.write(reinterpret_cast<const char*>(&s_out), sizeof(s_out));
    }

    student_record s_in{0, 0};
    {
        ifstream fin("student.bin", ios::binary);
        fin.read(reinterpret_cast<char*>(&s_in), sizeof(s_in));
    }

    cout << "Read from binary file → Roll: " << s_in.roll << ", Marks: " << s_in.marks <<
endl;
    return 0;
}

/*
Expected Deterministic Output:
Read from binary file → Roll: 105, Marks: 97
*/
```

# Conceptual Questions — Module 17: File Handling

**Q1.** Distinguish between `ios::app` and `ios::ate` file opening modes. **Q2.** Differentiate between `seekg()`, `seekp()`, `tellg()`, and `tellp()`.

# Debugging Questions — Module 17

**Q4.** Why is writing a class containing a `std::string` or `virtual` function directly to disk using `fout.write(reinterpret_cast<char*>(&obj), sizeof(obj))` a serious bug?

# Predict the Output Questions — Module 17

**Q3.** Why does `while (!fin.eof())` process the last record twice?

# Programming Questions — Module 17

**Q5.** Write a C++23 program that copies the contents of one text file into another while counting the total number of words.

# Detailed Solutions — Conceptual Questions (Module 17)

- **Q1 Solution**: `ios::app` forces **every** write operation to occur at the end of the file regardless of `seekp()` calls; `ios::ate` only positions the file pointer at the end upon opening, allowing subsequent `seekp()` calls to modify earlier bytes.
- **Q2 Solution**: `seekg` / `tellg` manipulate/query the input (**get**) position pointer, whereas `seekp` / `tellp` manipulate/query the output (**put**) position pointer.

## Detailed Solutions — Predict the Output & Debugging (Module 17)

- **Q3 Solution**: `eofbit` is only set *after* an input operation attempts to read past EOF and fails. Use `while (fin >> var)` instead.
- **Q4 Solution**: Classes with `std::string` store a heap pointer inside the object, and polymorphic classes store a `vptr`; raw byte dumping writes memory addresses that are invalid when read back in another process. Only trivially copyable structs/classes can use raw `read` / `write`.

# Detailed Solutions — Programming Questions (Module 17)

CPP

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
    ifstream fin("source.txt");
    ofstream fout("dest.txt");
    string word;
    int count = 0;
    while (fin >> word) {
        fout << word << " ";
        count++;
    }
    cout << "Words copied: " << count << endl;
    return 0;
}
```